Visual Cast Defect Inspection

Oct 20, 2026 · @NiteshKumar

MTAI Project · Team 4

Team member	Roll number
Ganesh Bhavaraju	114124024
Kruthika	114124056
Naqiya	114124064
Nitesh	114124066

Abstract

Visual Cast Defect Inspection is a phone-based app that inspects metal castings from a camera photo, flags defective parts, names the likely defect type and gives the operator pre-written corrective actions. It runs entirely on the device: the models execute in the browser, so no image leaves the phone and no GPU or server is needed.

The app offers three inspection modes. **Base model** uses deep-learning classifiers: a whole-part defect/OK classifier with a Grad-CAM heat map, and a new close-up classifier that names eight defect classes (gas porosity, shrinkage, crack, misrun, flash/burr, mould or sand defect, scratch, good surface). **Reference images** learns a new part from a handful of photos of good parts and flags anything unlike them, with no training. **Reference CAD** compares the photo with the part's CAD model, so it detects wrong parts, blocked or extra holes and missing or extra material in millimetres, and never mistakes a designed hole for a defect.

The models were trained and tested on public datasets from Kaggle and Roboflow, including real images captured on a foundry inspection line, and on synthetic photos rendered from CAD. On held-out test photos the close-up classifier reaches 90.4% accuracy (macro-F1 0.83) and misses 0.6% of defects. The CAD mode passes 97 to 100% of good parts at a fixed station and types 139 of 139 detected validation defects correctly.

1. Introduction

Casting is one of the oldest and most widely used ways to make metal parts: molten metal is poured into a mould and left to solidify into the shape of the cavity. Pump impellers, valve bodies, brackets, housings, connecting rods and engine blocks are all commonly

cast. The process is cheap for complex shapes, but it is also sensitive to small changes in sand moisture, pouring temperature, gating design and handling, and every foundry produces some defective parts.

1.1 Casting defects

A casting defect is any unwanted irregularity in the finished part. The common families are:

- **Gas defects:** blowholes, pinholes and porosity, caused by trapped gas from moist sand, cores or a gassy melt.
- **Shrinkage defects:** rough, jagged cavities where the metal contracted while solidifying and was not fed by a riser.
- **Cracks and hot tears:** fractures caused by contraction stresses when the mould or core resists shrinking.
- **Pouring defects:** misruns and cold shuts, where the metal froze before filling the mould or two streams did not fuse.
- **Mould material defects:** sand inclusions, scabs and swells from loose or weak moulding sand.
- **Finishing and handling defects:** flash and burrs along the parting line, chipped edges, scratches and blocked holes from missing or broken cores.

A defective part that reaches the customer can leak, crack in service or fail an assembly, and a whole order can be rejected because of a few bad parts.

1.2 Problem statement

Most small and medium foundries still inspect castings by eye. Manual inspection is slow, tiring and inconsistent: two inspectors grade the same part differently, accuracy drops over a shift, and there is no record of why a part was rejected. Commercial machine-vision systems exist, but they need dedicated cameras, lighting rigs, a large set of labelled defect images and often a GPU server, which puts them out of reach for most shops.

The problem this project addresses is: **build an inspection tool that a foundry can use with only a phone, that works for new part types without collecting thousands of defect images, and that tells the operator what the defect is and what to do about it.**

1.3 Objectives

1. Detect defective castings from a phone photo, on the device, with no server or GPU.
2. Support new part types quickly, either from a few photos of good parts or from the part's CAD model.

3. Never flag designed features such as holes as defects, and detect when the wrong part or the wrong face is shown.
4. Name the likely defect type and show where it is on the part.
5. Give pre-written causes, corrective actions and a disposition for each defect type.
6. Produce an inspection record: a PDF report per part and a CSV log for the shift.

2. Literature review

Automated casting inspection has moved from hand-tuned image processing to deep learning. Four lines of work are relevant.

- **Supervised classification and detection.** Networks such as VGG16, ResNet50 and Vision Transformers reach 99% or more defect/OK accuracy on the Kaggle impeller dataset. YOLO detectors (DES-YOLO, SLGA-YOLO, SFMW-YOLO) localise and name blowholes, cracks, porosity and sand holes. Every new part or defect class needs hundreds of labelled images.
- **Unsupervised anomaly detection.** PatchCore (Roth et al., CVPR 2022) stores patch features of good images from an ImageNet network and scores each test patch by its distance to the nearest stored patch, reaching state-of-the-art results on MVTec AD with no defect images. It cannot tell which differences matter, such as background or lighting, and does not name the defect.
- **CAD-based and pose-aware inspection.** PAD/MAD and OmniposeAD, SplatPose and Looking3D compare a photo with the object rendered or reconstructed from the camera's viewpoint, so designed features are handled correctly. Most need a calibrated multi-view rig or a GPU.
- **Foundry practice.** Defects are grouped into gas, shrinkage, pouring, mould material and metallurgical families and diagnosed by appearance: smooth round cavities indicate gas, rough dendritic ones shrinkage, rounded unfilled edges a misrun, bright sharp fractures breakage. Each family has well-known causes and fixes.

Gap addressed. Supervised models name defects but need large labelled sets; anomaly detection needs no defects but cannot name them; CAD methods understand geometry but need special hardware. This project combines all three in one phone app, adds defect typing to the two training-free modes and links every type to foundry guidance.

3. Datasets

The project uses three kinds of data: real industrial casting images from Kaggle, real labelled defect photos from Roboflow, and synthetic photos rendered from CAD models for testing the training-free modes.

3.1 Sources

Dataset	Source	Images	Used for
Casting product image data: impellers on a foundry inspection line, defective or OK	Kaggle (Pilot Technocast foundry, Rajkot)	7,348 + 1,300 originals	Whole-part model; good-surface crops
casing_defect_detection: boxes on blowholes, shrinkage, scratches, mould defects, burrs	Roboflow, CC BY 4.0	1,004	Close-up model
casting_dataset_kaggle: blowholes, shrinkage, cracks, misruns, cold shots, flash, scars, scratches	Roboflow, CC BY 4.0	781	Close-up model
casting-defects and casting-dataset-2: blow holes, cracks, burrs, scratches, shrinkage, porosity	Roboflow, CC BY 4.0	281	Close-up model
SIFA: real sand castings in a foundry	Kaggle	1,080	Good-surface crops
CAD renders of 5 parts, good and defective	This project	649 + test kits	Testing tabs 2 and 3

The Roboflow projects are hand annotations of the same Kaggle industrial impeller photos, so the close-up defect crops come from real castings photographed on a real inspection line.

3.2 Preparing the close-up defect set

The Roboflow sets label each defect with a bounding box. Each box was turned into a square crop centred on the defect, 1.6 times the box size, which is what a phone close-up of that defect looks like. Preparation steps:

1. **De-duplication.** Roboflow exports contain augmented copies of each photo (three per photo in one set). Copies were removed by matching the original file name.
2. **Class merging.** Equivalent labels from different projects were merged: BlowHoles, Blowholes, blow hole and Porosity became *gas*; Misrun and Cold shot became *misrun*; Burr, Flash and Plash became *flash/burr*; MouldDefect and Scar became *mould/surface*.
3. **Good surface.** Crops of the same sizes were cut from defect-free impellers and from SIFA sand castings.

4. **Split by photo.** Train, validation and test (70/15/15) were split by source photo, so crops from one photo never appear on both sides.
5. **Rejected data.** One Kaggle set with 12 defect classes turned out to be synthetic (black shapes painted onto one impeller photo) and was not used. One Roboflow project with misaligned boxes was also excluded.

Class	Train	Validation	Test
Flash / burr	2,318	556	599
Good surface	1,718	375	289
Gas porosity / blowhole	810	208	177
Shrinkage	741	149	159
Scratch	728	156	184
Mould / sand defect	427	90	80
Crack	123	32	26
Misrun / cold shut	119	26	20

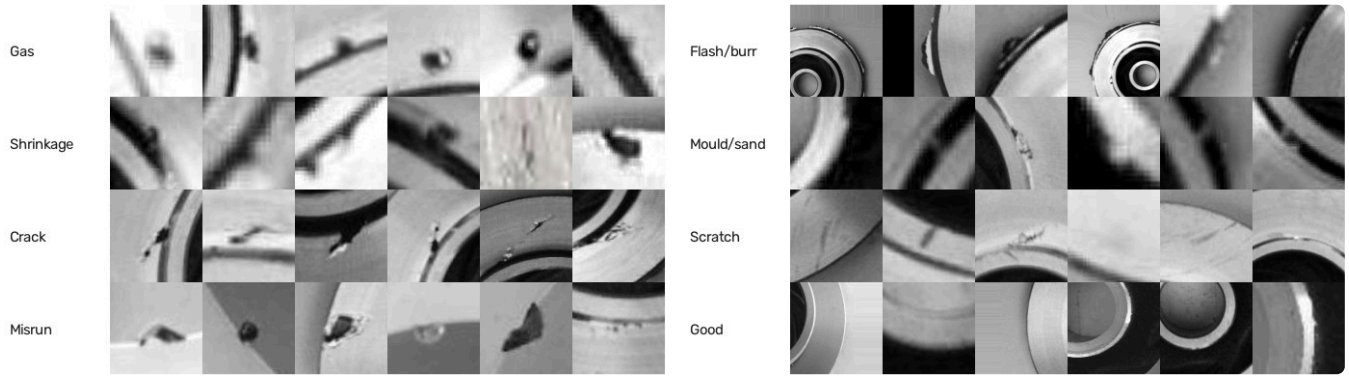


Figure 1. Close-up training crops, six per class, cut from the labelled defect boxes and from defect-free parts.

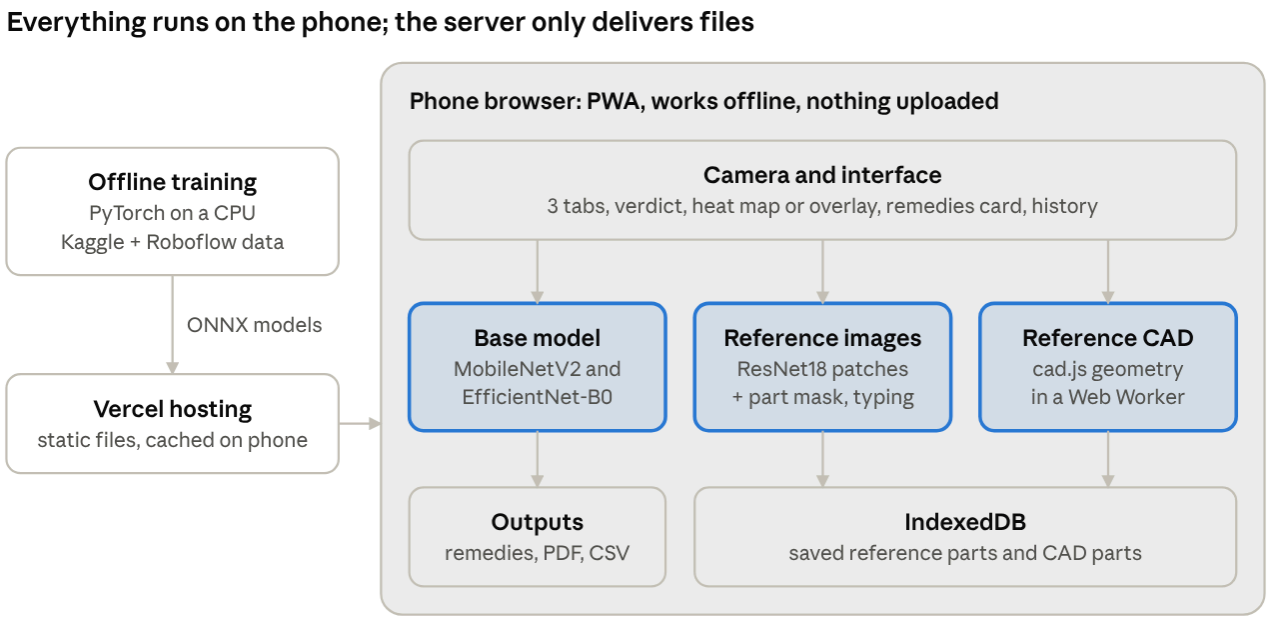
3.3 Synthetic test photos

Real photos of the same part with known, controlled defects are hard to collect, so the training-free modes were tested on photo-realistic renders. Parametric CAD parts (an impeller with 6 blades and 3 balance holes, an L-bracket, a pump housing, a connecting rod and a 6004-2RS bearing) were built in code, together with defective variants: through blowholes, blind pits, blocked holes, chipped rims, broken blades, cracks, flash, misruns and shrinkage cavities. Each was rendered in a headless browser with a physically based

metal material, a procedural sand-cast surface, a coloured mat, random rotation, position, tilt and lighting, then given sensor noise, blur and JPEG compression. The renderer is deliberately different from the one the app uses internally, so the tests are not circular.

4. System architecture

The app is a Progressive Web App (PWA): one web page that installs on a phone like an app, works offline after the first visit, and runs every model on the device. Nothing is uploaded; the server only delivers static files.



System architecture · 3 inspection engines on the phone

Figure 2. The models are trained offline and delivered once as static files; inspection, storage and reporting all happen in the phone's browser.

4.1 Technology stack

Layer	Technology
Interface	HTML, CSS and plain JavaScript; Web Worker for heavy work
Models in the browser	ONNX Runtime Web 1.20 (WebAssembly)
Geometry engine	`cad.js`, written for this project, no dependencies
Storage and offline	IndexedDB, localStorage, service worker
Reports	html2canvas and jsPDF

Layer	Technology
Training	Python, PyTorch, timm, ONNX export (CPU only)
Testing	Node.js, Playwright (headless Chromium), three.js, manifold3d
Hosting	Vercel static hosting from GitHub, a preview link per branch

4.2 Why on-device

Running in the browser means the app needs no GPU, no server costs and no network on the shop floor, and photos of customer parts never leave the phone. Models are kept small enough for a phone: the whole-part classifier is 8.9 MB, the close-up classifier 16 MB and the reference-image backbone 11 MB. Inspections take about 0.15 s to 1 s per photo on a laptop CPU.

4.3 Inspection flow

1. The camera frame (or an uploaded photo) is centre-cropped to a square and resized to 448 × 448 pixels.
2. The active tab's engine inspects it: a neural classifier (Base model), a patch comparison against good parts (Reference images) or a comparison with the CAD (Reference CAD).
3. The result is a verdict (PASS, REJECT, NO MATCH or WRONG PART), a heat map or overlay showing where the evidence is, and for rejects a list of typed findings.
4. The defect types are mapped to pre-written remedies, shown on screen and added to the PDF report.
5. Every inspection is logged with its time, verdict, scores and defect types, exportable as CSV.

5. Tab 1: Base model

The Base model tab uses trained neural networks and asks what the operator is photographing: a **close-up of a defect** (the new defect-type classifier) or the **whole impeller** (the defect/OK classifier).

5.1 Whole-part classifier with Grad-CAM

A MobileNetV2 network pretrained on ImageNet was fine-tuned on the Kaggle impeller photos to output two classes, defective and OK. Training used 224 × 224 inputs, random flips, rotations up to 20° and brightness/contrast jitter, the Adam optimiser at a learning rate of 3×10^{-4} and 4 epochs. A part is rejected when the defect probability exceeds 0.5.

To show *why* a part was rejected, the app draws a Grad-CAM heat map. Because the network ends in global average pooling followed by one linear layer, the gradient of the defect score with respect to each feature map is simply that class's weight, so the heat map can be computed on the phone from the last feature maps without back-propagation:

```
M(x, y) = \mathrm{ReLU}\Big(\sum_k w_k^{\text{defect}} \, A_k(x, y)\Big)
```

where A_k is the k-th feature map of the last convolution layer and w_k the linear weight for the defect class. The training script checks that this matches autograd Grad-CAM (cosine similarity above 0.99) and that the exported ONNX model gives the same outputs as PyTorch. On the 715 test photos of the Kaggle dataset the classifier is **99.7% accurate**: 451 of 453 defective parts are caught and none of the 262 good parts is rejected.

5.2 Close-up defect-type classifier

The whole-part model only says defect or OK. To name the defect, a second network classifies a close-up photo in which the defect fills the frame into eight classes: gas porosity/blowhole, shrinkage, crack, misrun/cold shut, flash/burr, mould or sand defect, scratch and good surface.

Model. EfficientNet-B0 with ImageNet weights (from the timm library), input 128 × 128. EfficientNet-B0 also ends in pooling and a linear layer, so the same weight-based heat map shows where the predicted type was seen.

Grayscale input. All training photos are grayscale, so the app converts phone photos to grayscale before classifying. This removes colour as a cue the model never learned, which narrows the gap between training photos and phone photos.

Training. 6,984 training crops (Section 3.2), 12 epochs, AdamW with weight decay 0.02, a one-cycle learning-rate schedule peaking at 6×10^{-4}, batch size 48. The rare classes (crack and misrun have about 120 crops each, flash/burr 2,318) are weighted in the loss by the square root of the inverse class frequency, with label smoothing of 0.1. Augmentation: rotation through 360° (a defect has no up or down), random crops of 60 to 100% of the frame, flips, brightness and contrast jitter and occasional blur. The epoch with the best validation macro-F1 is kept. Training took about 50 minutes on a 4-core CPU with no GPU.

Decision rule. A part is rejected when the probability of *good surface* is below 0.5. The defect type is the most likely of the seven defect classes, and the app reports how clearly it wins. When the top type holds less than 55% of the defect probability, the second-most likely type is named too ("could also be shrinkage").

Export. The model is exported to ONNX (16 MB) together with a JSON file of class names, input size, normalisation and the classifier weights used for the heat map. The export is checked against PyTorch on test images.

6. Tab 2: Reference images

The Reference images tab inspects any part type without training. The operator photographs 1 to 20 good parts once; every later photo is compared with them, and anything that looks unlike all of them is flagged. It follows the PatchCore idea (Section 2.2), adapted for a phone.

6.1 Enrolment

1. **Patch features.** Each good photo goes through a ResNet18 backbone pretrained on ImageNet (11 MB, ONNX). Intermediate feature maps are pooled into a 28 × 28 grid of 128-value patch descriptors, so each patch describes a small neighbourhood of the part.
2. **Rotations.** For round parts the operator ticks "any rotation" and each photo is also added at 90°, 180° and 270°.
3. **Memory bank.** All patches are stored in one bank, sub-sampled to at most 30,000 patches to keep it phone-sized.
4. **Threshold.** Each of the first five good photos is scored against the bank built from the others (leave-one-out). The largest score becomes the threshold, so the threshold reflects how much good parts naturally vary. A Stricter/Looser slider scales it between 0.6× and 1.4×.
5. **Saved.** The profile (bank, threshold, thumbnails, six reference photos for defect typing) is stored in the phone's IndexedDB.

6.2 Inspection

For a new photo, each of its 784 patches is matched to its nearest neighbour in the bank (a small ONNX distance model processes the bank in chunks of 6,000). The distance map is the heat map. The anomaly score is the mean of the 10 largest patch distances:

```
\text{score} = \frac{1}{10} \sum_{i \in \text{top 10}} \min_{b \in
\text{bank}} \lVert p_i - b \rVert_2
```

The verdict is REJECT if the score exceeds the threshold. A defect affects a few patches, while a different object differs almost everywhere, so if the *median* patch distance exceeds 1.35 times that of good parts the verdict is NO MATCH ("this is not the enrolled part").

6.3 Ignoring the background and designed holes

A plain patch comparison treats every pixel alike. A through-hole shows whatever the part is lying on, and a new table, sheet of paper or shadow made good parts look defective: in a test with good bearings on a different background, only 2 of 12 passed. The app now separates the part from its background before scoring:

- **Part mask.** On a coloured mat, pixels are classified by how much of the mat's hue they carry relative to their lightness, then split with Otsu's threshold; grey metal scores near zero whether lit or shaded. On white, grey or black paper, the mask comes from brightness difference against the border, with a morphological opening that removes ruled lines and paper texture.
- **Repainting.** Everything outside the part, including what shows through its holes and a 2-pixel band at its edge, is repainted with one flat colour: the typical background colour of the enrolled photos. This is done in enrolment and inspection alike.

The part's own appearance is all that remains to compare. New holes and chips still show, because background-coloured paint appears where good parts have metal, and a blocked hole shows metal where good parts have paint. With repainting, 7 of 12 good bearings on other backgrounds pass, all 16 defective bearings are still flagged, and detection on the impeller and bracket sets is unchanged or slightly better (Section 10).

6.4 Naming the defect type

A patch distance says *where* a part differs, not *how*. To name the defect, the app compares the flagged areas with the best-matching good reference photo:

1. **Registration.** The test photo and each of the six kept references are unrolled into polar images around the part's centre (360 angles × 40 rings of grey level and of the part mask, the mask weighted higher so holes decide between symmetric rotations). Circular cross-correlation finds the rotation, the median outline radius gives the scale, and a small shift search refines the position. The best-matching reference is used.
2. **Differences.** The reference is warped onto the test photo, and inside each hot area of the heat map the app measures metal lost, metal gained, holes opened or filled, new dark marks and dark structure that has disappeared.
3. **Rules.** Metal lost inside the part is a through blowhole (round) or through crack (long and thin). Metal lost at the outline is a chipped edge, or a misrun when large. Metal gained in a reference hole is a blocked hole; beyond the outline, flash (long strip) or

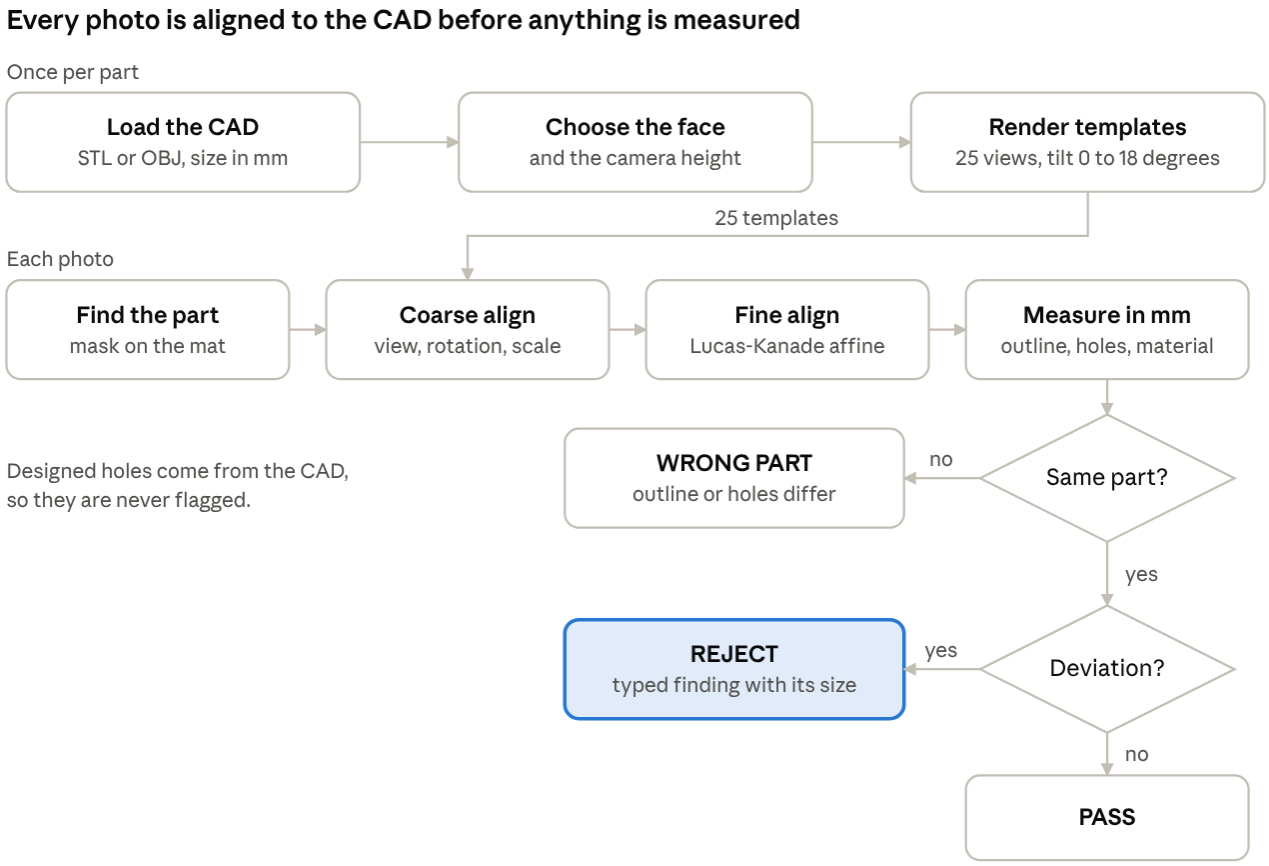
extra metal (compact). A new thin dark line is a crack; new dark pits are a surface blowhole or gas porosity (three or more). Missing dark structure is a missing or deformed feature (for example a broken blade).

4. **Filled holes.** Any designed hole that is filled is reported even if the heat map peaked elsewhere.

Findings are labelled *likely* and numbered on the heat map. Typing needs a background the part stands out from; otherwise the app says so and gives no type.

7. Tab 3: Reference CAD

The Reference CAD tab needs no photos of good parts: the operator loads the part's CAD file once, and every photo is checked against what the CAD says the part should look like from the camera. Because the CAD contains the designed holes, they are never mistaken for defects, and a different casting or the wrong face shown to the camera is reported as WRONG PART. The engine is written from scratch in JavaScript (`cad.js`, about 860 lines, no libraries) and runs in a Web Worker.



Reference CAD pipeline · setup once, then 4 steps and 2 decisions per photo

Figure 3. The CAD inspection pipeline. A wrong part is caught before any defect is reported, so a different casting is never mislabelled as defective.

7.1 Setting up a part

1. **Load the CAD.** STL (binary or text) or OBJ. The app reports the triangle count and size in millimetres.
2. **Pick the face** that will face the camera from six small previews (top, bottom, front, back, left, right).
3. **Enter the camera height** above the table in millimetres, so the templates use the right perspective (tall parts look different from close up).
4. **Template rendering.** The engine renders the chosen face with its own z-buffer rasteriser from 25 viewpoints: straight down and tilted 6°, 12° and 18° in 8 directions, at 320 × 320 pixels. Each template stores the silhouette, the designed through-holes and their mouths, the CAD edges with their directions, and the outline. This takes about 1 to 1.5 seconds and is cached.

7.2 Inspecting a photo

1. **Segmentation.** The part is separated from a plain coloured mat using the hue-projection rule of Section 6.3, followed by Otsu's threshold, morphological opening and closing, and the largest connected component. Tiny specks of background inside the part are filled; real holes are kept.
2. **Coarse alignment.** The photo mask and every template are reduced to 96 × 96, and rotation (2° steps), scale and position are searched. Among near-equal candidates the one whose hole pattern matches best is kept, which resolves symmetric parts.
3. **Fine alignment.** A Lucas-Kanade affine refinement on blurred masks aligns the photo with the template to sub-pixel accuracy.
4. **Measurements** in the template frame, in millimetres:
   - outline match (intersection over union of photo and CAD silhouettes);
   - feature ratio: edge strength across the CAD's interior edges against edge strength elsewhere on the part;
   - for each designed hole, how much background shows through its mouth;
   - openings in the photo where the CAD has none;
   - material missing or extra beyond a 1 mm tolerance band around the outline.

7.3 Decision rules

Check	Rule	Result if it fails
Outline	Outline match below 90%	WRONG PART
Features	Feature ratio below 1.45	WRONG PART

Check	Rule	Result if it fails
Hole pattern	At least half of the checkable holes blocked, or extra openings larger than half the designed hole area	WRONG PART
Designed holes	A checkable hole less than 50% open	REJECT: blocked hole
Extra openings	Opening of 2 mm² or more where the CAD has none	REJECT: hole not in the CAD
Outline material	Missing or extra area beyond the 1 mm band of at least 6 mm² (0.2% of the part area for small parts, never under 2 mm²)	REJECT: missing or extra material

A deep or off-centre hole may show almost no background from the camera position. A hole is checked only when at least 45% of its mouth should be see-through from that view; otherwise the result says how many holes could not be checked rather than guessing.

7.4 Naming the defect type

Each finding is typed from where it lies relative to the CAD and its shape in millimetres. Shape is measured from second moments of the region as an equivalent ellipse (length, width, elongation) and how much of that ellipse it fills. See-through fragments within 2.5 mm are grouped, because a crack or ragged void shows the background only in pieces.

Finding	Shape	Type reported
Opening not in the CAD	elongation ≥ 3 and width under 3 mm	Through crack, length
Opening not in the CAD	several fragments, or ragged	Shrinkage cavity (through)
Opening not in the CAD	round	Blowhole (or pinhole under 2 mm), with diameter
Designed hole	under 15% open, or metal in its mouth	Hole not formed (core missing or broken)
Designed hole	partly open	Partly blocked hole (sand or flash in it)
Material missing at the outline	150 mm² or more, or 25 mm or longer	Broken or unfilled section (misrun)

Finding	Shape	Type reported
Material missing at the outline	smaller	Chipped edge, size
Material beyond the outline	elongation ⩾ 4	Flash / fin, length and how far it sticks out
Material beyond the outline	compact	Extra metal (swell or lump)

The largest finding is listed first. The overlay draws the CAD edges in red, the designed holes in blue and the findings filled, aligned on the photo.

8. Defect remedies and inspection records

Finding a defect is only useful if the operator knows what to do next. Every defect type the three tabs can report is mapped to one of 13 pre-written guides, stored in `remedies.js`.

8.1 The remedy guides

Each guide has four parts, written from standard foundry practice:

- **Confirm:** what the defect looks like, so the operator can check the call by eye (for example, a blowhole has smooth, round walls; shrinkage is rough and dendritic).
- **Likely causes** in the process.
- **Fix the process:** concrete corrective actions.
- **This part:** the disposition (re-inspect with dye penetrant or X-ray, repair, re-work or scrap).

Guide	Main corrective actions
Gas porosity / blowhole	Drier sand and cores, more vents, degas the melt, pour through a filter
Shrinkage cavity	Resize or move risers, exothermic sleeves, chills
Crack / hot tear	Collapsible cores, fillets at corners, controlled cooling
Misrun / cold shut	Hotter, faster, uninterrupted pour; larger gates; vents
Chipped or broken edge	Gentler shake-out, saw off gates, padded racks

Guide	Main corrective actions
Blocked or missing hole	Core-setting check, stronger cores, better decoring
Flash / fin / burr	Clamp moulds, repair patterns, raise die clamp force
Swell / extra metal	Even ramming, mould coating, lower pouring height
Sand or slag inclusion	Blow out moulds, ceramic filters, skim slag
Scratch or handling mark	Trays and separators, check tools and blasting media
Missing or deformed feature	Follow the breakage or misrun guide; check pattern and core box
Surface anomaly	Clean and re-inspect; mould coating for sand burn-on
Untyped defect	Identify the type by eye or with another tab

8.2 On screen

When a part is rejected, a "What to do" card appears under the verdict with the guide for the defect type found. If several types were found, chips switch between them.

8.3 PDF report and CSV log

Each inspection can be exported as a one- or two-page PDF quality report:

- **Page 1:** document number, part ID, date and time, verdict stamp, confidence gauge, the evidence zone on the part, the captured image beside the heat map or CAD overlay, a defect-type table, the recommended action, the decision rule and model used, and sign-off lines.
- **Page 2:** the action plan, with each defect type's causes, fixes and disposition.

The CSV log lists every inspection of the session with time, part type, verdict, defect probability, anomaly score, threshold, defect types and findings, ready for a spreadsheet or quality system.

9. User interface and implementation

The interface is built for one-handed use on the shop floor:

- **Navigation:** a floating, rounded "liquid glass" bar switches between the three tabs with a sliding highlight. Only GPU-friendly transform animations are used, so tab switches stay near 50 frames per second on a slow phone.
- **Camera and result:** the live camera with corner brackets showing exactly what the model sees, beside the heat map or CAD overlay. On a phone the result shrinks to a picture-in-picture corner that enlarges on tap.
- **Verdict and remedies:** PASS in green, REJECT in red, NO MATCH or WRONG PART in amber, one line of detail, and the remedies card for rejects.
- **History:** the last ten inspections with thumbnail, defect type and a PDF button each.
- **Style:** dark theme with maroon highlights, Poppins font, translucent glass panels. The app asks for camera access on first open; gallery photos can be tested too.

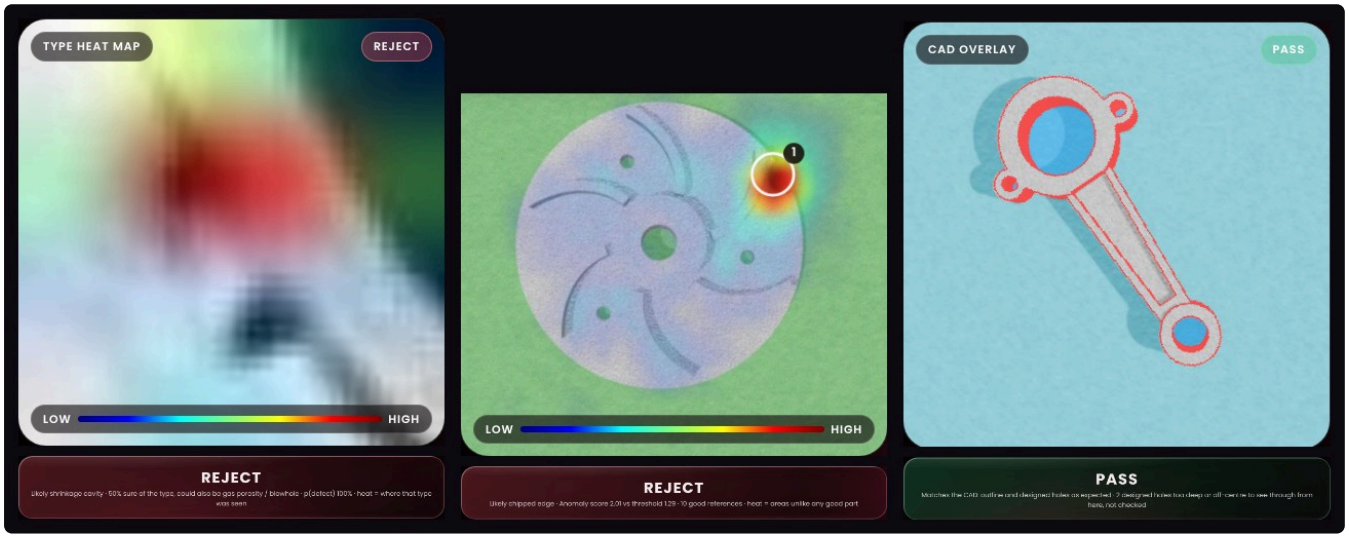


Figure 4. The three tabs in use. Left: Base model, close-up mode, with the heat map of where the type was seen and the likely type plus runner-up. Middle: Reference images, difference map against enrolled good parts with the typed finding numbered. Right: Reference CAD, CAD edges (red) and designed holes (blue) aligned on a connecting rod.

Implementation. The page is about 1,150 lines (`index.html`), the geometry engine 860 (`cad.js`) and the remedies 100 (`remedies.js`), with no front-end framework. CAD rendering, CAD inspection and defect typing run in a Web Worker built from the same `cad.js`, which also runs under Node.js for automated tests. Saved parts live in IndexedDB; a service worker serves the page network-first and caches models for offline use. Each Git branch gets its own Vercel preview link and the main branch is the production app.

10. Testing and results

All three tabs were tested on photos they were never tuned on. The close-up classifier was evaluated on held-out photos, and the training-free tabs on synthetic test kits with known defects. Every change was also checked in the real app with automated browser tests.

10.1 Base model

The whole-part classifier is 99.7% accurate on the 715 Kaggle test photos (2 of 453 defects missed, 0 of 262 good parts rejected).

The close-up defect-type classifier was tested on 1,534 crops from photos it never saw in training. It is **90.4% accurate** with a macro-F1 of 0.83. As a defect detector it misses 8 of 1,245 defects (0.6%) and flags 4 of 289 good surfaces (1.4%).

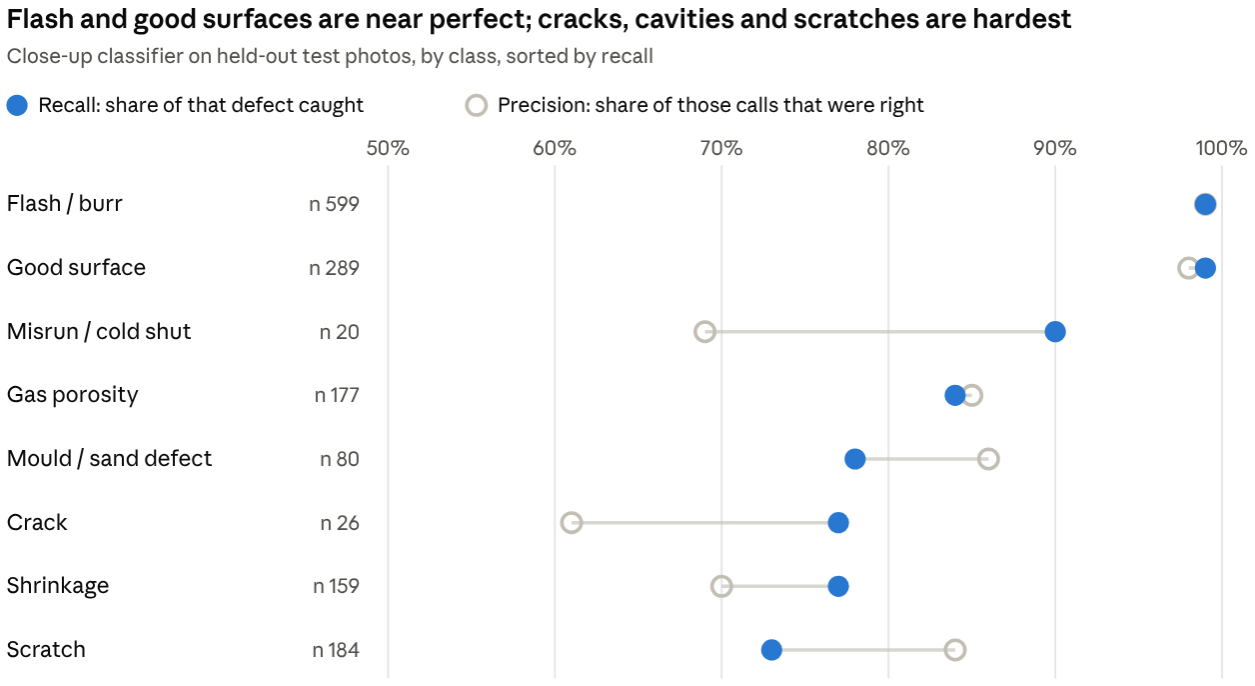


Figure 5. Recall and precision per class on the held-out test photos. The x axis starts at 50%.

The main confusion is between the two cavity types: 20 gas crops were called shrinkage and 21 shrinkage crops were called gas. The source annotators are not consistent between these two either. In the real app each prediction takes about 0.15 s, and 22 of 24 held-out crops sampled through the user interface were typed correctly.

10.2 Reference CAD

The CAD engine was validated on 649 rendered photos of four parts, each with good parts, defective variants and wrong parts.

Part	Good parts passing at a fixed station
Impeller (6 blades, 3 balance holes)	99%
L-bracket	100%
Pump housing	97%
Connecting rod	100%

Every detected validation defect was given the right type: **139 of 139** (through blowholes 46, blocked or uncored holes 47, chipped edges 46). On extra shapes, flash was typed correctly 8 of 8 times and misruns 8 of 8. Through cracks were detected 4 of 8 times (2 called crack). Shrinkage voids were detected 5 of 8 times and called blowholes, because only their round core shows the mat through.

Test kit	Result
Connecting rod blind kit (12 photos, never tuned on)	9 of 12 correct in the app
6004-2RS bearing: good	10 of 10 pass
Bearing: lump on the rim / bore blocked	4 of 4 / 4 of 4 flagged
Bearing: piece broken out / crack through the ring	3 of 4 / 3 of 4 flagged
Bearing: washer or larger 6005 bearing shown instead	2 of 4 / 1 of 4 called WRONG PART

A photo takes about 0.8 s and a new CAD part about 1 to 1.5 s to prepare.

10.3 Reference images

The tab was tested on rendered station photos with fixed lighting: 10 good parts enrolled, then good and defective parts inspected.

Test set	Good parts passing	Defects flagged
Impeller station (64 defects, 10 types)	2 of 6	55 of 64
Blind impeller station	4 of 4	23 of 32
Blind bracket station	4 of 4	15 of 18
Bearing on 6 other backgrounds	7 of 12 (2 of 12 before background repainting)	16 of 16

The heat-map peak lay on the true defect in 53 of 60 photos, measured against twin renders without the defect. Defect typing named the right type in 50 of 64 photos on the development set and 42 of 50 on the blind sets. Blocked holes, broken blades, misruns and shrinkage were typed reliably; hairline cracks and blind pits were the hardest.

10.4 Automated app tests

Browser tests in headless Chromium drove the real app at desktop and phone sizes: camera permission and the no-camera path, all three tabs end to end (enrolment, CAD import, inspection by file upload), history, PDF report and CSV export. Tab animation stayed smooth under 6× CPU throttling (27 to about 49 frames per second after optimisation).

11. Limitations and future work

11.1 Limitations

- **One product family for training.** Every labelled defect photo shows the same grayscale impeller, with defects about 50 pixels across. Close-ups of other castings will be less accurate until the model is retrained with some of the user's own photos.
- **Hard classes.** Gas porosity and shrinkage are often confused; crack and misrun had only about 120 training examples each. Corrosion and seal damage are outside the eight classes.
- **CAD mode sees outline and through-holes only.** Surface cracks, pits, chips that do not go through the part's height and changes to raised interior features are not judged there.
- **No absolute scale.** A phone photo has no scale reference, so a geometrically similar part (a 6005 bearing in place of a 6004) can pass the CAD check.

- **Background and validation.** The CAD tab needs a coloured mat, and a silver part on grey paper of similar brightness cannot be separated. The training-free tabs were validated mostly on rendered photos.

11.2 Future work

1. Retrain the close-up classifier with 20 to 30 labelled close-ups per class from a partner foundry, and add corrosion and seal-damage classes.
2. Add a printed scale marker on the mat so the CAD mode can measure absolute size.
3. Run the Reference images surface check inside the CAD outline for parts that have both.
4. Design a fixed station with a ring light and stand, and add STEP import and two-sided inspection.
5. Share inspection history across phones for a supervisor dashboard.

12. Conclusion

Visual Cast Defect Inspection shows that useful casting inspection can run on an ordinary phone, with no GPU, no server and no images leaving the device. The three tabs cover the three situations a foundry meets:

- **Base model:** a trained model for a known product. The whole-part model is 99.7% accurate on its test set, and the new close-up model names eight defect classes at 90.4% accuracy.
- **Reference images:** a new part learned from a few good photos, now robust to the background and to what shows through designed holes.
- **Reference CAD:** a part checked against its CAD model. Designed holes are never flagged, wrong parts are caught, and deviations are measured in millimetres.

All three tabs name the likely defect type, show where it is, and link it to pre-written causes, corrective actions and a disposition. The inspection ends as a PDF report and a CSV log.

The main next step is real-world data: a few hundred labelled close-ups from a partner foundry would retrain the close-up classifier for that foundry's parts, and real photos with known defects would confirm the synthetic validation of the two training-free tabs.

13. References

1. K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, P. Gehler. *Towards Total Recall in Industrial Anomaly Detection* (PatchCore). CVPR 2022.

2. R. R. Selvaraju et al. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.* ICCV 2017.
3. M. Sandler et al. *MobileNetV2: Inverted Residuals and Linear Bottlenecks.* CVPR 2018.
4. M. Tan, Q. Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.* ICML 2019.
5. K. He et al. *Deep Residual Learning for Image Recognition* (ResNet). CVPR 2016.
6. B. D. Lucas, T. Kanade. *An Iterative Image Registration Technique with an Application to Stereo Vision.* IJCAI 1981.
7. N. Otsu. *A Threshold Selection Method from Gray-Level Histograms.* IEEE Trans. SMC, 1979.
8. Q. Zhou et al. *PAD: A Dataset and Benchmark for Pose-agnostic Anomaly Detection* (MAD, OmniposeAD). NeurIPS 2023. arXiv:2310.07716
9. M. Kruse et al. *SplatPose & Detect: Pose-Agnostic 3D Anomaly Detection.* CVPR Workshops 2024. arXiv:2404.06832
10. *Looking 3D: Anomaly Detection with 2D-3D Alignment.* CVPR 2024. arXiv:2406.19393
11. *DES-YOLO: a novel model for real-time detection of casting surface defects.* PeerJ Computer Science, 2024. link
12. *SLGA-YOLO: A Lightweight Castings Surface Defect Detection Method.* Sensors, 2024. link
13. R. Dabhi. *Casting product image data for quality inspection.* Kaggle (Pilot Technocast, Rajkot). link
14. Roboflow Universe casting defect datasets (CC BY 4.0): casing_defect_detection, casting_dataset_kaggle, casting-defects, casting-dataset-2.
15. *Sandcast Images for Foundry Automation (SIFA).* Kaggle. link
16. ONNX Runtime Web, Microsoft. onnxruntime.ai
17. R. Wightman. *PyTorch Image Models (timm).* GitHub